

Frontend: Theory Assignment 3

1. What is the difference between frontend and backend JavaScript?

Frontend JavaScript	Backend JavaScript
Runs in the user's web browser.	Runs on the server.
Used to create interactive web pages.	Used to handle server-side operations.
Can manipulate HTML and CSS using the DOM.	Can interact with databases and files.
Examples: Chrome, Firefox, Edge.	Commonly uses Node.js.
Mainly handles user interface and interactions.	Mainly handles business logic, APIs, authentication, etc.

2. What is DOM?

DOM (Document Object Model) is a programming interface that represents an HTML document as a tree of objects.

JavaScript can use the DOM to **access, modify, add, or remove HTML elements and their styles**.

Example:

```
document.getElementById("heading").textContent = "Hello World";
```

3. What is an event in JavaScript?

An **event** is an action or occurrence that happens in a web page and can be detected and handled by JavaScript.

Examples of events are:

- click
- mouseover
- keydown
- submit
- load

Example:

```
button.addEventListener("click", function() {  
    alert("Button clicked!");  
});
```

4. What is Event Loop in JavaScript?

The **Event Loop** is a mechanism that allows JavaScript to handle asynchronous operations while JavaScript itself is single-threaded.

It continuously checks whether the **Call Stack** is empty and whether there are callbacks waiting in the task queue. When the stack is empty, it moves the waiting callback to the Call Stack for execution.

Example:

```
console.log("Start");  
  
setTimeout(() => {  
    console.log("Timeout");  
}, 0);  
  
console.log("End");
```

Output:

```
Start  
End  
Timeout
```

5. What is Hoisting in JavaScript?

Hoisting is JavaScript's behavior of processing declarations before executing the code in their scope.

Example with var:

```
console.log(x);  
var x = 10;
```

Output:

undefined

Function declarations are also hoisted:

```
sayHello();  
  
function sayHello() {  
    console.log("Hello");  
}
```

`let` and `const` are also hoisted, but they cannot be accessed before their declaration because they remain in the **Temporal Dead Zone (TDZ)**.

6. What is Event Bubbling?

Event bubbling is the process in which an event starts from the target element and then propagates upward through its parent elements.

Example:

```
<div id="parent">  
    <button id="child">Click Me</button>  
</div>  
  
<script>  
document.getElementById("child").addEventListener("click", function()  
{  
    console.log("Button clicked");  
});  
  
document.getElementById("parent").addEventListener("click", function()  
{  
    console.log("Parent clicked");  
});  
</script>
```

When the button is clicked, the event first occurs on the **button** and then bubbles up to the **parent**.

7. What is the difference between Named Function, Anonymous Function and Arrow Function?

Named Function

A function that has a specific name.

```
function add(a, b) {  
    return a + b;  
}
```

Anonymous Function

A function without a name, usually assigned to a variable or passed as a callback.

```
const add = function(a, b) {  
    return a + b;  
};
```

Arrow Function

A shorter syntax for writing functions.

```
const add = (a, b) => {  
    return a + b;  
};
```

Main difference: Named functions have a name, anonymous functions do not have their own name, and arrow functions provide a shorter syntax and have different **this** behavior.

8. Explain for-of and for-in loop.

for-of

The for-of loop is used to iterate over the **values** of an iterable such as an array or string.

```
let fruits = ["Apple", "Mango", "Banana"];

for (let fruit of fruits) {
  console.log(fruit);
}
```

Output:

```
Apple
Mango
Banana
```

for-in

The for-in loop is used to iterate over the **enumerable property keys** of an object.

```
let student = {
  name: "Rahul",
  age: 20
};

for (let key in student) {
  console.log(key, student[key]);
}
```

Output:

```
name Rahul
age 20
```

9. Explain any five array methods.

1. *push()*

Adds one or more elements to the end of an array.

```
let arr = [1, 2];  
arr.push(3);
```

```
console.log(arr);
```

Output:

```
[1, 2, 3]
```

2. *pop()*

Removes the last element from an array.

```
let arr = [1, 2, 3];  
arr.pop();
```

```
console.log(arr);
```

Output:

```
[1, 2]
```

3. *map()*

Creates a new array by applying a function to every element.

```
let numbers = [1, 2, 3];
```

```
let result = numbers.map(x => x * 2);
```

```
console.log(result);
```

Output:

```
[2, 4, 6]
```

4. *filter()*

Creates a new array containing elements that satisfy a condition.

```
let numbers = [1, 2, 3, 4];

let result = numbers.filter(x => x > 2);

console.log(result);
```

Output:

```
[3, 4]
```

5. *reduce()*

Reduces all elements of an array to a single value.

```
let numbers = [1, 2, 3, 4];

let sum = numbers.reduce((total, x) => total + x, 0);

console.log(sum);
```

Output:

```
10
```

10. What is Callback Method?

A **callback** is a function that is passed as an argument to another function and is executed later or when a particular operation is completed.

Example:

```
function greet(name, callback) {
    console.log("Hello " + name);
    callback();
}
```

```
function message() {  
    console.log("Welcome!");  
}
```

```
greet("Rahul", message);
```

Output:

Hello Rahul

Welcome!

Here, `message` is passed as a **callback function** to the `greet()` function.